

CyberShield AI — Source Code Documentation

Technical Codebase Architecture & Developer Guide

Team Lead: Kota Bhanu Prasanth Reddy (KBPR-2026) **Problem Statement:** Honeywell-2026-Q4 (Software)

Deployed Website: <https://cybershield-ai-1-5gyk.onrender.com/>

GitHub Repository: <https://github.com/Prasanth4689/CyberShield-AI>

1. Repository Structure & Directory Tree

```
CyberShield-AI/
├── ml/                                # Core Machine Learning Modules
│   ├── data_generator.py             # Synthetic access-log generator (59.5k events, 200 entities)
│   ├── feature_engineering.py        # 43-feature pipeline (temporal, geo-velocity, device)
│   ├── baseline_model.py             # Tier 1: Isolation Forest + One-Class SVM profiler
│   ├── detection_model.py            # Tier 2: XGBoost + LSTM Autoencoder ensemble
│   ├── classifier.py                 # Tier 3: SMOTE + Multi-Class XGBoost attack classifier
│   ├── explainer.py                  # Tier 4: SHAP TreeExplainer & NL explanation generator
│   ├── evaluate.py                   # Metrics computation (Confusion Matrix, ROC, PR)
│   └── train_pipeline.py              # End-to-end training orchestrator
├── backend/                           # Flask REST API Server
│   ├── app.py                        # API endpoints + static dashboard server (no-cache headers)
│   └── config.py                     # Centralized configuration & directory paths
├── dashboard/                         # Analyst-Facing Web Application
│   ├── index.html                    # Multi-page SPA (Overview, Analytics, Alerts, Model)
│   ├── css/styles.css                # Glassmorphism dark/light theme CSS
│   ├── js/api.js                     # REST API client with mock fallback
│   ├── js/charts.js                  # Chart.js visualization engine
│   └── js/app.js                     # Tab router, spotlight search (Ctrl+K), theme, events
├── datasets/                          # Generated access logs & processed feature vectors
├── models/                           # Saved model checkpoints (.joblib)
├── reports/figures/                   # High-res evaluation plots (PNG)
├── run.py                             # CLI entry point (train / serve)
├── Procfile                          # Render deployment start command
└── render.yaml                       # Render cloud deployment blueprint
```

2. Core Machine Learning Modules (ml/)

`ml/data_generator.py`

Generates 59,566 synthetic access events for 200 entities (150 users, 30 service accounts, 20 edge devices) over 30 days. Generates habitual behavioral profiles (preferred login hours, consistent geo locations, typical

resources) for 97-98% normal traffic, and injects 7 attack patterns (2-3% total).

ml/feature_engineering.py

`FeatureEngineer` class with `fit_transform(df)` method. Engineers 43 behavioral features: temporal (login counts 1h/6h/24h, off-hours flag), geo-velocity (km/h between consecutive logins), resource overlap ratio (Jaccard similarity), device fingerprint changes, and entity age baselines.

ml/baseline_model.py

`BaselineProfiler` : Combines Isolation Forest (contamination=0.02) and One-Class SVM (nu=0.02) trained exclusively on normal traffic. Handles cold-start entities (<20 events) using population-level fallback baselines.

ml/detection_model.py

`AnomalyDetector` : Ensemble of XGBoost binary classifier (tuned with `scale_pos_weight=50`) and a 2-layer LSTM Autoencoder (64 units) trained on event sequences. Returns a continuous Risk Score (0–100).

ml/classifier.py

`AttackClassifier` : Multi-class XGBoost model with SMOTE oversampling to categorize anomalous events into 7 attack types: *brute_force*, *impossible_travel*, *credential_stuffing*, *lateral_movement*, *device_spoofing*, *low_and_slow_exfiltration*, *insider_drift*.

ml/explainer.py

`AlertExplainer` : SHAP TreeExplainer calculating per-event feature attributions. Converts top positive/negative SHAP contributions into natural-language narratives for SOC analysts.

3. Backend REST API Endpoints (backend/app.py)

ENDPOINT	METHOD	DESCRIPTION
/	GET	Serves main dashboard SPA index.html
/api/stats	GET	Returns high-level KPI metrics (total events, anomalies, FPR, mean risk)
/api/alerts	GET	Paginated & filtered alert queue (attack_type, entity_type, search)
/api/alerts/<id>	GET	Full alert detail including SHAP feature attributions & explanation
/api/entities	GET	List of active entities ranked by average risk score
/api/predict	POST	Inference endpoint to score new access log events in real time
/api/model/metrics	GET	Model evaluation metrics (Accuracy, F1, Confusion Matrix, ROC)
/api/reports/export	GET	Exports filtered incident alerts as a downloadable CSV file
/api/health	GET	Health check endpoint returning system status

4. Web Dashboard Architecture (dashboard/)

Multi-Page View Router (dashboard/js/app.js)

Implements client-side view switching without page reloads across 4 sections: `#page-overview`, `#page-analytics`, `#page-alerts`, and `#page-model`. Syncs with browser URL hash (`#overview`, `#analytics`, `#alerts`, `#model`) and handles back/forward history navigation.

Global Spotlight Search (Ctrl+K / Cmd+K)

Search overlay implemented with inline display logic for 100% reliable rendering. Filters across all alerts by entity ID, IP address, location, and attack taxonomy with keyboard navigation (↑ Enter Esc).

Dual Theme System (dashboard/css/styles.css)

Full light/dark theme support using CSS custom properties (`--bg-primary`, `--bg-card`, `--glass-border`). Theme preference persists in `localStorage`.

5. Local Setup & Deployment Guide

Production Cloud Deployment: Live on Render PaaS at <https://cybershield-ai-1-5gyk.onrender.com/> with Gunicorn WSGI server and Python 3.11 runtime.

```
# 1. Clone repository
git clone https://github.com/Prasanth4689/CyberShield-AI.git
cd CyberShield-AI

# 2. Install dependencies
pip install -r requirements.txt

# 3. Train ML Pipeline (Generates data & models)
python run.py train

# 4. Start Local Dashboard & API Server
python run.py serve
# Open http://localhost:5000 in your browser
```